

CORE 09 — Programming in C and C++ / OOP

Shared-core file 9 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points),

10/15-mark answer (worked explanation + code snippet with expected output).

0. Why this matters — where C/C++ appears in your three papers

Elective	Where	How heavy
CS Degree	P-I §1 "Basics and Programming — Programming in C/C++: Data types, declarations and expressions, functions, pointers, arrays."	Heavy
Computer Forensic	TP-II §4 — a full unit : "Elements of C ... Structured data types — arrays, struct, union, string, pointers. OOP Concepts : class, object, inheritance, polymorphism, overloading. C++ : tokens, datatypes, operators, control statements, functions, parameter passing, class and objects, constructors and destructor, overloading, inheritance, templates, exception handling."	Very heavy
CS Diploma	P-II §5 "Programming in C" — history, algorithms, flowcharts, operators & hierarchy, I/O (<code>printf/scanf/getchar/putchar/getch/putch</code>), control statements, loops, and (per the source PDF) arrays, functions, pointers, structures, file handling.	Heavy

All three electives test C; two of them test C++/OOP. The Diploma is strong on **C fundamentals and I/O**; the Forensic paper is strong on **the full OOP feature set**; the Degree paper on **pointers, arrays and functions**. Study the union once. **Every code snippet in an answer should state its expected output** — examiners reward a correctly predicted output more than prose.

Topic checklist — C

- ☐ Features & history; program structure; compilation stages
- ☐ Tokens: keywords, identifiers, constants, strings, operators, separators
- ☐ Data types and their **sizes**; type modifiers; type conversion (implicit/explicit)
- ☐ **Operators + full precedence/associativity table**
- ☐ I/O: `printf`, `scanf`, `getchar`, `putchar`, `getch`, `putch` + **format specifiers**
- ☐ Control: `if`, `if-else`, nested `if`, `switch`, `goto`
- ☐ Loops: `for`, `while`, `do-while`, `break`, `continue`
- ☐ Arrays & strings; **string functions**
- ☐ Functions: **call by value vs reference**, recursion, **storage classes table**
- ☐ **Pointers**: arithmetic, pointers & arrays, pointer-to-pointer, function pointers
- ☐ **malloc / calloc / realloc / free** differences
- ☐ **Structures vs unions**
- ☐ File handling
- ☐ Preprocessor and macros

Topic checklist — C++ / OOP

- ☐ Four pillars: encapsulation, abstraction, inheritance, polymorphism
- ☐ Classes & objects; access specifiers
- ☐ **Constructors (default/parameterised/copy) & destructors**
- ☐ Function overloading & **operator overloading**
- ☐ **Inheritance types + diamond problem + virtual base classes**

- [] **Virtual functions, vtable, runtime polymorphism**
- [] Compile-time vs run-time polymorphism
- [] Abstract classes / pure virtual functions
- [] Templates (function & class)
- [] Exception handling
- [] Friend functions; static members; `this` pointer; inline functions

PART A — THE C LANGUAGE

1. Features, history and program structure

Concept

C was developed by **Dennis Ritchie at Bell Labs in 1972** to write the **UNIX** operating system. It is a **general-purpose, procedural, structured, middle-level** language — "middle" because it combines **high-level** readability with **low-level** access to memory (pointers, bit operations), making it ideal for system software, compilers, embedded systems and OS kernels. Standardised as **ANSI C (C89/C90)**, then **C99, C11, C17**.

Key features (list these for a 5-marker): portable, fast/efficient, structured (functions & blocks), rich set of operators and built-in functions, **pointers** (direct memory access), **dynamic memory allocation**, recursion, extensible (user libraries), small core with a large standard library, case-sensitive.

"**Middle-level**" is the single most examined descriptor — say it.

1.1 Structure of a C program

```
#include <stdio.h>           /* 1. Preprocessor directive (header) */
#define PI 3.14159           /* 2. Symbolic constant / macro      */

int square(int x);           /* 3. Function prototype (declaration) */

int main(void) {              /* 4. main() - execution starts here */
    int n = 5;                /* Local declarations */
    printf("%d\n", square(n)); /* Statements */
    return 0;                 /* Return status to the OS */
}

int square(int x) {           /* 5. Function definition */
    return x * x;
}
```

Expected output: 25

Compilation stages (worth a diagram): **source (.c)** → **Preprocessor** (expands `#include`, `#define`) → **Compiler** (produces assembly/object `.o`) → **Assembler** → **Linker** (joins object files + libraries) → **executable (.exe / a.out)** → **Loader** (loads into memory to run).

Likely exam questions

- [5] State any six features of C. Why is it called a middle-level language?
- [5] Explain the structure of a C program with an example.
- [5] Explain the stages of compilation of a C program.

MCQ traps

Trap	Correct answer
"Who developed C and when?"	Dennis Ritchie, 1972, Bell Labs.
"C was written to develop?"	The UNIX OS.
"C is a ____ level language?"	Middle level.
"Execution of a C program begins at?"	main() .
" #include is processed by the?"	Preprocessor.

2. Tokens — keywords, identifiers, constants

Concept

A **token** is the smallest individual unit of a C program. There are **six** kinds: **keywords, identifiers, constants, string literals, operators, and special symbols (separators).**

Keywords — 32 reserved words in ANSI C (`int`, `float`, `if`, `else`, `while`, `for`, `return`, `struct`, `union`, `const`, `static`, `void`, `sizeof`, ...). They have fixed meaning and **cannot be used as identifiers.**

Identifiers — names given to variables, functions, arrays, etc. **Rules:**

1. Begin with a **letter or underscore** (`_`), never a digit.
2. Followed by letters, digits or underscores only.
3. **No spaces, no special symbols, no keywords.**
4. **Case-sensitive** (`Sum` $\neq$ `sum`).

Constants (literals): integer (`75`, `0x4B` hex, `075` octal), floating (`3.14`, `2.5e3`), character (`'A'` — stored as its ASCII value), string (`"NPSC"` — an array of chars ending in `'\0'`). A constant can be fixed with `const` or `#define`.

Escape sequences: `\n` newline, `\t` tab, `\0` null, `\\` backslash, `\'` quote, `\"`, `\r`, `\b`, `\a` bell.

MCQ traps

Trap	Correct answer
"Number of keywords in ANSI C?"	32.
"Valid identifier: <code>2sum</code> , <code>_sum</code> , <code>su m</code> , <code>for</code> ?"	<code>_sum</code> (others break a rule).
"A character constant is stored as its?"	ASCII value.
"A string ends with?"	The null character <code>\0</code> .
"Is C case-sensitive?"	Yes.

3. Data types, sizes and type conversion

Concept

C data types classify variables by the kind and size of value they hold.

Category	Types
----------	-------

Basic (primary)	<code>char</code> , <code>int</code> , <code>float</code> , <code>double</code> , <code>void</code>
Derived	array, pointer, function
User-defined	<code>struct</code> , <code>union</code> , <code>enum</code> , <code>typedef</code>

Type modifiers: `signed`, `unsigned`, `short`, `long` change range/size.

3.1 Typical sizes (32/64-bit systems) — MCQ material

Type	Size (bytes)	Format specifier	Typical range
<code>char</code>	1	<code>%c</code>	−128 to 127 (signed)
<code>unsigned char</code>	1	<code>%c</code>	0 to 255
<code>short int</code>	2	<code>%hd</code>	−32,768 to 32,767
<code>int</code>	4 (usually)	<code>%d</code>	−2.1×10 ⁹ to 2.1×10 ⁹
<code>unsigned int</code>	4	<code>%u</code>	0 to 4.29×10 ⁹
<code>long int</code>	4 or 8	<code>%ld</code>	platform-dependent
<code>long long int</code>	8	<code>%lld</code>	large
<code>float</code>	4	<code>%f</code>	~6–7 significant digits
<code>double</code>	8	<code>%lf</code>	~15–16 significant digits
<code>long double</code>	10/12/16	<code>%Lf</code>	extended

Use `sizeof(type)` to get the exact size on a given machine — never assume.

3.2 Type conversion

Implicit (type promotion / coercion) — the compiler auto-converts in a mixed expression, promoting toward the "wider" type: `char` → `int` → `unsigned` → `long` → `float` → `double` → `long double`. Example: `int / int` truncates, but `int / float` promotes the int to float.

Explicit (type casting) — the programmer forces a conversion: `(float)5 / 2` = 2.5, whereas `5 / 2` = 2 (integer division). **Integer division truncates toward zero** — the classic trap.

```
#include <stdio.h>
int main(void){
    int a = 5, b = 2;
    printf("%d\n", a / b);           /* integer division */
    printf("%.2f\n", (float)a / b); /* cast → real division */
    return 0;
}
```

Expected output:

```
2
2.50
```

MCQ traps

Trap	Correct answer
"Size of <code>int</code> (typical)?"	4 bytes (2 on old 16-bit compilers).

"Size of <code>double</code> ?"	8 bytes.
" <code>5 / 2</code> gives?"	2 (integer division).
"Specifier for <code>double</code> in scanf?"	<code>%lf</code> .
"Operator to find size at compile time?"	<code>sizeof</code> .
"Widest basic type?"	<code>long double</code> .

4. Operators — with the precedence/associativity table

Concept

C has a rich operator set:

Category	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (<code>%</code> = modulo, integers only)
Relational	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>
Logical	<code>&&</code> <code> </code> <code>!</code> (short-circuit)
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> ...
Increment/decrement	<code>++</code> <code>--</code> (pre and post)
Conditional (ternary)	<code>cond ? a : b</code>
Others	<code>sizeof</code> , <code>&</code> (address), <code>*</code> (dereference), <code>.</code> , <code>-></code> , <code>,</code> (comma), <code>[]</code> , <code>()</code>

4.1 Precedence and associativity table (high → low)

Level	Operators	Associativity
1 (highest)	<code>()</code> <code>[]</code> <code>.</code> <code>-></code> (postfix <code>++</code> <code>--</code>)	Left → right
2	unary <code>!</code> <code>~</code> <code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>*(deref)</code> <code>&(addr)</code> <code>sizeof</code> (cast)	Right → left
3	<code>*</code> <code>/</code> <code>%</code>	Left → right
4	<code>+</code> <code>-</code>	Left → right
5	<code><<</code> <code>>></code>	Left → right
6	<code><</code> <code><=</code> <code>></code> <code>>=</code>	Left → right
7	<code>==</code> <code>!=</code>	Left → right
8	<code>&</code> (bitwise AND)	Left → right
9	<code>^</code> (bitwise XOR)	Left → right
10	<code> </code> (bitwise OR)	Left → right
11	<code>&&</code>	Left → right
12	<code> </code>	Left → right

13	<code>?:</code> (ternary)	Right → left
14	<code>= += -= ...</code> (assignment)	Right → left
15 (lowest)	<code>,</code> (comma)	Left → right

The three facts examiners test: (1) **unary, ternary and assignment are right-associative**, everything else in the middle is left-associative; (2) *** / % beat + -**;
(3) **arithmetic beats relational beats logical beats assignment**.

Worked evaluation. `int x = 5 + 3 * 2;` → * first → `5 + 6 = 11` (not 16).

`int y = 10 > 5 && 2 < 1;` → relational first → `1 && 0 = 0`.

4.2 Pre- vs post-increment (a guaranteed MCQ)

```
#include <stdio.h>
int main(void) {
    int a = 5, b;
    b = a++;      /* post: assign THEN increment → b=5, a=6 */
    printf("a=%d b=%d\n", a, b);
    int c = 5, d;
    d = ++c;      /* pre: increment THEN assign → d=6, c=6 */
    printf("c=%d d=%d\n", c, d);
    return 0;
}
```

Expected output:

```
a=6 b=5
c=6 d=6
```

Rule: post-increment uses the value first, then increments; pre-increment increments first, then uses.

MCQ traps

Trap	Correct answer
<code>"5 + 3 * 2 = ?"</code>	11 (* before +).
<code>"% works on?"</code>	Integers only (not float).
<code>"Which operators are right-associative?"</code>	Unary, ternary <code>?:</code>, assignment .
<code>"a = b = 5 evaluates?"</code>	Right to left (assignment associativity).
<code>"i++ returns?"</code>	The old value, then increments.
<code>"Ternary operator?"</code>	?: — the only ternary operator in C.

5. Input / output and format specifiers

Concept

C I/O is via the **standard library** `<stdio.h>` (formatted) and `<conio.h>` (console, Turbo C).

Function	Purpose	Header
----------	---------	--------

<code>printf</code>	Formatted output to screen	<code>stdio.h</code>
<code>scanf</code>	Formatted input from keyboard (needs <code>&</code> for variables)	<code>stdio.h</code>
<code>getchar</code>	Read a single char (echoed , needs Enter)	<code>stdio.h</code>
<code>putchar</code>	Write a single char	<code>stdio.h</code>
<code>getch</code>	Read a char without echo, no Enter needed	<code>conio.h</code>
<code>putch</code>	Write a char to console	<code>conio.h</code>
<code>gets</code> / <code>puts</code>	Read/write a string (whole line)	<code>stdio.h</code>

`getch()` vs `getchar()` — the classic distinction: `getch()` reads a key **immediately**, **without echoing it** to the screen (used for "Press any key..."); `getchar()` **echoes** and waits for Enter.

5.1 Format specifiers

Specifier	Data type
<code>%d / %i</code>	int (decimal)
<code>%u</code>	unsigned int
<code>%f</code>	float / double (printf)
<code>%lf</code>	double (scanf)
<code>%c</code>	char
<code>%s</code>	string
<code>%x / %X</code>	hexadecimal
<code>%o</code>	octal
<code>%e</code>	scientific notation
<code>%p</code>	pointer address
<code>%%</code>	a literal <code>%</code>

Width/precision: `%5d` (min width 5), `%-5d` (left-justified), `%.2f` (2 decimals), `%6.2f` (width 6, 2 decimals).

```
#include <stdio.h>
int main(void){
    int n; float pi = 3.14159f;
    printf("Enter a number: ");
    scanf("%d", &n);          /* & is essential */
    printf("n = %d, pi = %.2f\n", n, pi);
    return 0;
}
```

If input is `7`, **expected output:** `n = 7, pi = 3.14`

The `&` in `scanf` is compulsory for basic variables (it passes the address so `scanf` can write back) — omitting it is the most common runtime crash and a favourite MCQ.

MCQ traps

Trap	Correct answer
"scanf needs which operator before a variable?"	<code>&</code> (address-of).
" <code>getch()</code> differs from <code>getchar()</code> how?"	No echo, no Enter needed.
"Specifier to print a string?"	<code>%s</code> .
" <code>%.2f</code> prints?"	Float with 2 decimal places .
" <code>getch/putch</code> are declared in?"	<code>conio.h</code> .
"Print a literal percent sign?"	<code>%%</code> .

6. Control statements and loops

Concept — decision making

Statement	Form
<code>if</code>	<code>if(cond){ ... }</code>
<code>if-else</code>	<code>if(cond){...} else {...}</code>
Nested if / else-if ladder	<code>if(c1){} else if(c2){} else {}</code>
<code>switch</code>	multiway branch on an integer/char constant
<code>goto</code>	unconditional jump to a label (discouraged)

switch rules (examined): the expression must be **integer or char** (not float, not string); each `case` label must be a **constant**; **break** is needed or execution **falls through** to the next case; **default** is optional and handles no-match.

```
#include <stdio.h>
int main(void){
    int day = 3;
    switch(day){
        case 1: printf("Mon\n"); break;
        case 2: printf("Tue\n"); break;
        case 3: printf("Wed\n"); break;
        default: printf("Other\n");
    }
    return 0;
}
```

Expected output: Wed. (Remove the `breaks` and it would print Wed Other — the fall-through trap.)

6.1 Loops

Loop	When the condition is tested	Runs at least once?
<code>for(init; cond; update)</code>	Before each iteration (entry-controlled)	No

<code>while (cond)</code>	Before each iteration (entry-controlled)	No
<code>do { } while (cond);</code>	After each iteration (exit-controlled)	Yes — at least once

do-while is the only exit-controlled loop — it always executes the body once even if the condition is false initially. That is the single most examined loop fact.

- **break** — exits the **innermost** loop/switch immediately.
- **continue** — skips the rest of the current iteration and goes to the next.

```
#include <stdio.h>
int main(void) {
    for(int i = 1; i <= 5; i++){
        if(i == 3) continue; /* skip 3 */
        if(i == 5) break;    /* stop before 5 */
        printf("%d ", i);
    }
    return 0;
}
```

Expected output: 1 2 4

Likely exam questions

- [5] Differentiate `while` and `do-while` loops with examples.
- [5] Explain the `switch` statement and the role of `break` and `default`.
- [5] Differentiate `break` and `continue`.
- [10] Explain the decision-making and looping statements in C with syntax and examples.

MCQ traps

Trap	Correct answer
"Which loop is exit-controlled?"	do-while (runs at least once).
" <code>switch</code> works on which types?"	int / char (not float or string).
"Missing <code>break</code> in switch causes?"	Fall-through .
" <code>continue</code> does?"	Skips to the next iteration .
" <code>break</code> exits?"	The innermost loop or switch.
"Entry-controlled loops?"	for and while .

7. Arrays and strings

Concept

An **array** in C is a fixed-size, contiguous collection of same-type elements, indexed from

0. `int a[5];` reserves 5 ints (`a[0]...a[4]`). 2-D: `int m[3][4];`. C does **no bounds checking** — accessing `a[5]` is undefined behaviour (a common exam warning).

A **string** in C is simply a **character array terminated by the null character `\0`**. `char s[6] = "HELLO";` uses 6 bytes (5 letters + `\0`). There is **no separate string type**.

7.1 String functions (<string.h>) — memorise

Function	Action
<code>strlen(s)</code>	Length excluding <code>\0</code>
<code>strcpy(d, s)</code>	Copy s into d
<code>strcat(d, s)</code>	Append s to d
<code>strcmp(a, b)</code>	Compare: 0 if equal , <0 if a<b if a>b (lexicographic)
<code>strncpy/strncat/strncmp</code>	Length-limited versions
<code>strrev(s)</code>	Reverse (Turbo C)
<code>strupr/strlwr</code>	Upper/lower case (Turbo C)
<code>strstr(s, t)</code>	Find substring t in s

```
#include <stdio.h>
#include <string.h>
int main(void){
    char a[20] = "Data";
    char b[] = "Structure";
    strcat(a, b);
    printf("%s (len %d)\n", a, strlen(a));
    printf("cmp=%d\n", strcmp("abc", "abd"));
    return 0;
}
```

Expected output:

```
DataStructure (len 13)
cmp=-1
```

(`strcmp` returns the sign of the difference of the first differing characters: 'c'-'d' = -1.)

MCQ traps

Trap	Correct answer
"Array indexing starts at?"	0 .
"A C string ends with?"	<code>\0</code> (null).
" <code>strlen("HELLO")</code> = ?"	5 (excludes <code>\0</code> , but storage needs 6).
" <code>strcmp</code> returns 0 when?"	Strings are equal .
"Does C check array bounds?"	No .
"String functions are in?"	<code><string.h></code> .

8. Functions — call by value/reference, recursion, storage classes

Concept

A **function** is a self-contained block that performs a task; it promotes **modularity, reusability and readability**. Three parts: **declaration (prototype), definition, and call**. A function may **return** one value (via `return`) and receive several **parameters**.

8.1 Call by value vs call by reference

	Call by value	Call by reference
What is passed	A copy of the argument's value	The address of the argument (a pointer)
Effect on original	Cannot change the caller's variable	Can modify the caller's variable
Default in C?	Yes — C is call-by-value	Simulated using pointers (<code>&</code> and <code>*</code>)
Overhead	Copies data	Passes an address (cheap for large data)

C is strictly call-by-value. "Call by reference" in C is *achieved by passing pointers* — this nuance is heavily examined. (True reference parameters `&` exist only in **C++**.)

```
#include <stdio.h>
void swapVal(int a, int b){ int t=a; a=b; b=t; }           /* no effect */
void swapRef(int *a, int *b){ int t=*a; *a=*b; *b=t; }     /* swaps */

int main(void){
    int x=10, y=20;
    swapVal(x, y); printf("value: x=%d y=%d\n", x, y);
    swapRef(&x, &y); printf("ref  : x=%d y=%d\n", x, y);
    return 0;
}
```

Expected output:

```
value: x=10 y=20
ref   : x=20 y=10
```

8.2 Recursion

A function that **calls itself** with a smaller input until a **base case** stops it (see also DataStructures §4.4). Every call adds a stack frame.

```
#include <stdio.h>
int fib(int n){
    if(n < 2) return n;           /* base case */
    return fib(n-1) + fib(n-2);   /* recursive case */
}
int main(void){ printf("%d\n", fib(6)); return 0; }
```

Expected output: 8 (sequence 0 1 1 2 3 5 8 → fib(6)=8).

8.3 Storage classes — the table

A **storage class** defines a variable's **scope** (visibility), **lifetime** (how long it lives), **default initial value**, and **storage location**.

Storage class	Keyword	Scope	Lifetime	Default value	Stored in
---------------	---------	-------	----------	---------------	-----------

Automatic	<code>auto</code>	Local (block)	Until block ends	Garbage	Stack
Register	<code>register</code>	Local (block)	Until block ends	Garbage	CPU register (if free)
Static	<code>static</code>	Local: block; retains value between calls . Global: file	Entire program	Zero	Data segment
External	<code>extern</code>	Global (across files)	Entire program	Zero	Data segment

`static` local variable — the exam favourite: it is **initialised once** and **retains its value between function calls**, unlike `auto`.

```
#include <stdio.h>
void counter(void){
    static int c = 0;    /* initialised once, persists */
    c++;
    printf("%d ", c);
}
int main(void){ counter(); counter(); counter(); return 0; }
```

Expected output: 1 2 3 (with `auto int c` it would print 1 1 1).

Likely exam questions

- **[5]** Differentiate call by value and call by reference with an example.
- **[5]** Explain the four storage classes in C with scope, lifetime and default value.
- **[5]** What is recursion? Write a recursive function for factorial.
- **[10]** Explain functions in C — declaration, definition, call, parameter passing methods — with the swap example, and describe storage classes in a table.

MCQ traps

Trap	Correct answer
"C passes arguments by?"	Value (reference simulated via pointers).
" <code>static</code> local variable default value?"	0 , and it retains value between calls .
" <code>auto</code> variable default value?"	Garbage .
"Fastest-access storage class?"	<code>register</code> .
" <code>extern</code> is used for?"	Global variables across files.
"Recursion needs a?"	Base case .

9. Pointers

Concept

A **pointer** is a variable that **stores the memory address** of another variable. `int *p = &a;` makes `p` point to `a`; `*p` (**dereference**) accesses the value at that address; `&a` (**address-of**) gives `a`'s address. Pointers give C its power: **dynamic memory, call-by-**

reference, efficient arrays/strings, data structures (linked lists, trees).

```
int a = 10;
int *p = &a;      /* p holds the address of a */
printf("%d", *p); /* prints 10 — the value AT that address */
```

9.1 Pointer arithmetic

Pointer arithmetic is **scaled by the size of the pointed-to type**. If `p` points to an `int` (4 bytes) at address 2000, then `p + 1` is **2004**, not 2001 — it moves to the next *element*. Valid: `++`, `--`, `+ n`, `- n`, and subtracting two pointers (gives the element count between them). **Invalid**: adding two pointers, multiplying/dividing pointers.

9.2 Pointers and arrays

An array name **is** a constant pointer to its first element: `a[i]` is exactly `*(a + i)`. So `a`, `&a[0]`, and `p = a` are equivalent starting points. This equivalence is examined constantly.

```
#include <stdio.h>
int main(void) {
    int a[] = {10, 20, 30};
    int *p = a;                /* points to a[0] */
    printf("%d %d\n", *(p+1), p[2]); /* 20 30 */
    return 0;
}
```

Expected output: 20 30

9.3 Pointer to pointer

A **double pointer** stores the address of a pointer: `int **pp = &p;`. `*pp` gives `p`, `**pp` gives the final value. Used for 2-D dynamic arrays and modifying a pointer inside a function.

9.4 Function pointers

A **function pointer** holds the address of a function, enabling **callbacks** and jump tables:

```
int add(int a, int b){ return a + b; }
int (*fp)(int, int) = add;      /* fp points to add */
printf("%d", fp(3, 4));        /* prints 7 */
```

9.5 Dynamic memory — malloc / calloc / realloc / free

Declared in `<stdlib.h>`. Memory is taken from the **heap** and **must be freed manually** — C has no garbage collector. Forgetting `free` causes a **memory leak**.

Function	Purpose	Initialises memory?	Arguments
<code>malloc(size)</code>	Allocate <code>size</code> bytes	No (contains garbage)	one: total bytes
<code>calloc(n, size)</code>	Allocate an array of <code>n</code> elements	Yes — all zeros	two: count, element size
<code>realloc(ptr, size)</code>	Resize a previously allocated block	Preserves old data	pointer, new size
<code>free(ptr)</code>	Release the block back to the heap	—	pointer

The examined differences: `malloc` leaves **garbage**, `calloc` **zero-initialises**;

`malloc` takes **one** argument, `calloc` takes **two**; both return a `void*` (cast if needed) and **NULL on failure** (always check).

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int *arr = (int*)calloc(3, sizeof(int)); /* {0,0,0} */
    arr[0] = 5; arr[1] = 10;
    arr = (int*)realloc(arr, 5 * sizeof(int)); /* grow to 5 */
    arr[4] = 99;
    printf("%d %d %d\n", arr[0], arr[1], arr[4]);
    free(arr); /* release */
    return 0;
}
```

Expected output: 5 10 99

Likely exam questions

- [5] What is a pointer? Explain `&` and `*` with an example.
- [5] Explain pointer arithmetic. Why is `p+1` not `address+1`?
- [5] Differentiate `malloc` and `calloc`.
- [10] Explain dynamic memory allocation in C with `malloc`, `calloc`, `realloc` and `free`, giving a program.
- [10] Explain the relationship between pointers and arrays with examples; explain pointer-to-pointer and function pointers.

MCQ traps

Trap	Correct answer
" <code>&</code> operator gives?"	The address of a variable.
" <code>*p</code> is the?"	Value at the address (dereference).
" <code>p + 1</code> for an int pointer moves by?"	sizeof(int) bytes (e.g. 4).
" <code>a[i]</code> is equivalent to?"	<code>*(a + i)</code> .
" <code>calloc</code> vs <code>malloc</code> ?"	<code>calloc</code> zero-initialises and takes two args.
"malloc returns on failure?"	NULL .
"Not freeing memory causes?"	A memory leak .
"Functions for dynamic memory are in?"	<code><stdlib.h></code> .

10. Structures vs unions

Concept

A **structure** (`struct`) groups **different-type** variables (members) under one name — a user-defined record. A **union** looks identical syntactically but **all members share the same memory location**.

Criterion	struct	union
Memory	Sum of all members' sizes (+ padding)	Size of the largest member only
Members active at once	All simultaneously	Only one at a time
Use	Store a full record (all fields together)	Save memory when only one field is used at a time
Changing one member	Others unaffected	Overwrites the shared memory

```
#include <stdio.h>
struct S { int i; char c; double d; }; /* 4 + 1 + 8 (+pad) */
union U { int i; char c; double d; }; /* size of double = 8 */
int main(void) {
    printf("struct=%lu union=%lu\n", sizeof(struct S), sizeof(union U));
    return 0;
}
```

Expected output (typical): struct=16 union=8 (struct padded to 16; union = largest member, 8). Access members with `.` on a variable and `->` on a pointer to it.

MCQ traps

Trap	Correct answer
"Union size equals?"	Its largest member.
"Struct size equals?"	Sum of members (plus padding).
"In a union, how many members hold valid data at once?"	One .
"Access member via a pointer?"	-> operator.
"Which saves memory?"	Union .

11. File handling

Concept

File handling lets a program store data **permanently** on disk. Uses a **FILE *** pointer (from `<stdio.h>`). Workflow: **fopen** → **read/write** → **fclose**.

Modes: "r" read, "w" write (truncate/create), "a" append, "r+" / "w+" / "a+" read-write; add "b" for binary ("rb", "wb").

Function	Purpose
<code>fopen(name, mode)</code>	Open; returns <code>NULL</code> on failure
<code>fclose(fp)</code>	Close and flush
<code>fprintf / fscanf</code>	Formatted write/read
<code>fputc / fgetc</code>	Char write/read
<code>fputs / fgets</code>	String write/read

<code>fwrite / fread</code>	Block (binary) write/read
<code>fseek / ftell / rewind</code>	Random access / position
<code>feof(fp)</code>	End-of-file test

```
#include <stdio.h>
int main(void){
    FILE *fp = fopen("out.txt", "w");
    if(fp == NULL){ printf("Cannot open\n"); return 1; }
    fprintf(fp, "NPSC 2026\n");
    fclose(fp);
    return 0;
}
```

Writes `NPSC 2026` to `out.txt`. Always check `fopen` for `NULL`.

MCQ traps

Trap	Correct answer
"File pointer type?"	<code>FILE *</code>
"Mode <code>"w"</code> on an existing file?"	Truncates it (overwrites).
"fopen returns on failure?"	<code>NULL</code> .
"Append mode?"	<code>"a"</code> .
"End-of-file check?"	<code>feof()</code> .

12. Preprocessor and macros

Concept

The **preprocessor** runs **before compilation**, acting on lines starting with `#`. It does **text substitution** — it does not know C syntax.

Directive	Purpose
<code>#include</code>	Insert a header file (<code><...></code> system, <code>"..."</code> user)
<code>#define</code>	Define a macro / symbolic constant
<code>#undef</code>	Remove a macro
<code>#ifdef / #ifndef / #endif</code>	Conditional compilation (include guards)
<code>#if / #elif / #else</code>	Conditional compilation on constant expressions
<code>#pragma</code>	Compiler-specific instruction

Object-like macro: `#define PI 3.14`. **Function-like macro:** `#define SQ(x) ((x)*(x))`.

The macro-parenthesis trap: `#define SQ(x) x*x` then `SQ(2+3)` expands to `2+3*2+3 = 11`, not 25. Always parenthesise: `#define SQ(x) ((x)*(x))` → `((2+3)*(2+3)) = 25`. Macros are

not functions — no type checking, no evaluation, just text replacement (and no semicolon).

```
#include <stdio.h>
#define SQ(x) ((x)*(x))
int main(void){ printf("%d\n", SQ(2+3)); return 0; }
```

Expected output: 25

MCQ traps

Trap	Correct answer
"Preprocessor runs?"	Before compilation.
" <code>#define</code> does?"	Text substitution .
" <code>SQ(x) x*x</code> , <code>SQ(2+3)</code> = ?"	11 (missing parentheses trap).
"Include guards use?"	<code>#ifndef</code> / <code>#define</code> / <code>#endif</code> .
" <code><stdio.h></code> vs <code>"myfile.h"</code> ?"	System header vs user header.

PART B — C++ AND OBJECT-ORIENTED PROGRAMMING

13. The four pillars of OOP

Concept

C++ (Bjarne Stroustrup, Bell Labs, **1979**, originally "C with Classes") extends C with **object-oriented** features. OOP models software as interacting **objects** — bundles of **data (attributes)** and **functions (methods)** — rather than procedures acting on data. Its four foundational principles:

Pillar	Meaning	Achieved by
Encapsulation	Binding data and the functions that operate on it into one unit (class) , and hiding internal state	Classes + private members
Abstraction	Exposing only the essential features , hiding implementation detail	Public interface; abstract classes; header/implementation split
Inheritance	A class (derived) reuses and extends another (base)	<code>class D : public B</code>
Polymorphism	" Many forms " — one interface, many behaviours	Overloading (compile-time), virtual functions (run-time)

Data hiding (via **private**) is the mechanism; **encapsulation** is the concept — a common MCQ distinction. **Procedural (C) vs OOP (C++)**: procedural is function-centred with global, exposed data; OOP is object-centred with data hidden inside objects and accessed through methods.

MCQ traps

Trap	Correct answer
"Who developed C++ / when?"	Bjarne Stroustrup, 1979 (Bell Labs).
"Wrapping data + functions into one unit?"	Encapsulation .
"Showing only essential features?"	Abstraction .
"Many forms?"	Polymorphism .
"Data hiding is enforced by?"	The private access specifier.
"C++ was originally called?"	"C with Classes" .

14. Classes, objects and access specifiers

Concept

A **class** is a **user-defined type** — a blueprint combining data members and member functions. An **object** is an **instance** of a class (memory is allocated only when an object is created). The class describes; the object exists.

Access specifiers control member visibility:

Specifier	Accessible from
private (default for class)	Only within the class and its friends
protected	Within the class and derived classes
public	Anywhere the object is visible

(A **struct** in C++ is a class whose members default to **public**.)

```
#include <iostream>
using namespace std;
class Rectangle {
    int w, h;                // private by default
public:
    void set(int a, int b){ w=a; h=b; }
    int area(){ return w*h; }
};
int main(){
    Rectangle r;             // object
    r.set(4, 5);
    cout << r.area() << endl;
    return 0;
}
```

Expected output: 20

MCQ traps

Trap	Correct answer
------	----------------

"Default access in a <code>class</code> ?"	private .
"Default access in a <code>struct</code> (C++)?"	public .
"An object is a(n)?"	Instance of a class.
" <code>protected</code> members are visible in?"	The class and derived classes .
"Memory is allocated when?"	An object is created (not at class definition).

15. Constructors and destructors

Concept

A **constructor** is a special member function with the **same name as the class**, **no return type** (not even `void`), **called automatically** when an object is created, used to initialise data. A **destructor** (`~ClassName`, no args, no return) is called automatically when the object is destroyed, to release resources.

Type of constructor	Signature	Purpose
Default	<code>Box()</code>	No parameters; sets defaults
Parameterised	<code>Box(int l, int w)</code>	Initialise with supplied values
Copy	<code>Box(const Box &b)</code>	Create a new object as a copy of an existing one

Copy constructor is invoked when: an object is initialised from another (`Box b2 = b1;`), an object is **passed by value**, or **returned by value**. If you don't write one, the compiler supplies a **shallow copy** — dangerous when the class holds pointers (both copies free the same memory → the reason to write a **deep-copy** constructor).

```
#include <iostream>
using namespace std;
class Box {
    int v;
public:
    Box(){ v = 0; cout << "default\n"; }           // default
    Box(int x){ v = x; cout << "param " << v << "\n"; } // parameterised
    Box(const Box &b){ v = b.v; cout << "copy " << v << "\n"; } // copy
    ~Box(){ cout << "destroy " << v << "\n"; }      // destructor
};
int main(){
    Box a;           // default
    Box b(7);        // parameterised
    Box c = b;       // copy
    return 0;
}
```

Expected output:

```
default
param 7
copy 7
```

```
destroy 7
destroy 7
destroy 0
```

(Destructors run in **reverse order** of construction: c, b, a.)

MCQ traps

Trap	Correct answer
"A constructor returns?"	Nothing — no return type.
"Constructor name equals?"	The class name .
"Copy constructor parameter?"	A reference (<code>const Box&</code>) — passing by value would recurse infinitely.
"Destructors run in what order?"	Reverse of construction.
"Can a constructor be overloaded?"	Yes (default, parameterised, copy).
"Default copy constructor does a?"	Shallow copy .

16. Function and operator overloading (compile-time polymorphism)

Concept

Overloading = the same name with **different parameter lists**, resolved by the compiler at **compile time (static / early binding)** — this is **compile-time polymorphism**.

Function overloading: several functions share a name but differ in **number or type of parameters** (return type alone is **not** enough to overload).

```
int add(int a, int b){ return a+b; }
double add(double a, double b){ return a+b; }
int add(int a, int b, int c){ return a+b+c; }
```

Operator overloading: give an existing operator (+, -, ==, <<, ...) new meaning for **user-defined types**, so objects can be used with natural syntax.

```
#include <iostream>
using namespace std;
class Complex {
    int re, im;
public:
    Complex(int r=0, int i=0): re(r), im(i) {}
    Complex operator+(const Complex &c){          // overload +
        return Complex(re + c.re, im + c.im);
    }
    void show(){ cout << re << "+" << im << "i\n"; }
};
int main(){
    Complex a(2,3), b(4,1);
    Complex c = a + b;    // calls operator+
    c.show();
    return 0;
}
```

```
}
```

Expected output: `6+4i`

Operators that **CANNOT** be overloaded (a guaranteed MCQ): `.` (dot), `::` (scope resolution), `?:` (ternary), `.*` (pointer-to-member), and `sizeof`.

MCQ traps

Trap	Correct answer
"Overloading is resolved at?"	Compile time (static/early binding).
"Can functions differ by return type alone?"	No — parameters must differ.
"Which operators can't be overloaded?"	<code>., ::, ?, .* sizeof</code> .
"Operator overloading works on?"	User-defined types (classes).
"Overloading is which polymorphism?"	Compile-time .

17. Inheritance, the diamond problem and virtual base classes

Concept

Inheritance lets a **derived class** reuse and extend a **base class**, modelling an "is-a" relationship (a Car *is a* Vehicle). Syntax: `class Derived : access Base { ... };`.

17.1 Types of inheritance

Type	Structure
Single	One base → one derived ($A \rightarrow B$)
Multiple	One derived, several bases ($A, B \rightarrow C$)
Multilevel	Chain ($A \rightarrow B \rightarrow C$)
Hierarchical	One base → several derived ($A \rightarrow B, A \rightarrow C$)
Hybrid	A combination of the above (often multiple + hierarchical → the diamond)

17.2 Mode of inheritance — how base access changes

Base member	<code>public</code> inheritance	<code>protected</code> inheritance	<code>private</code> inheritance
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	not inherited (inaccessible)	<code>inaccessible</code>	<code>inaccessible</code>

Private members of the base are never directly accessible in the derived class (only via the base's public/protected functions) — an examined point.

17.3 The diamond problem and virtual base classes

The **diamond problem** arises in **multiple/hybrid inheritance**: if class **D** inherits from both **B** and **C**, and both **B** and **C** inherit from a common base **A**, then **D** ends up with **two copies of A's members** — an ambiguity (which **A::x?**).



Solution: virtual base classes. Declare A as a **virtual base** in B and C:

`class B : virtual public A { ... };` and `class C : virtual public A { ... };`. Now D inherits **only one shared copy** of A, removing the ambiguity.

```
class A { public: int x; };
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };    // one shared A – no ambiguity
```

Likely exam questions

- [5] Define inheritance. Explain its types with diagrams.
- [5] Explain the visibility modes (public/protected/private inheritance).
- [10] What is the diamond problem? Explain how virtual base classes solve it, with code.
- [10] Explain the types of inheritance with syntax and an example each.

MCQ traps

Trap	Correct answer
"Diamond problem occurs in?"	Multiple / hybrid inheritance.
"Diamond problem is solved by?"	Virtual base classes .
"Are base's private members inherited?"	Not accessible in the derived class.
"Inheritance models which relationship?"	"is-a" .
"A → B → C is which type?"	Multilevel .
"One base, many derived?"	Hierarchical .

18. Virtual functions, vtables and run-time polymorphism

Concept

Run-time polymorphism = deciding **which function to call at run time**, based on the actual object type — **dynamic / late binding**. It is achieved with **virtual functions** through a **base-class pointer or reference**.

The mechanism: declare a member `virtual` in the base and **override** it in derived classes.

When you call it through a base pointer that actually points to a derived object, **the derived version runs**. Without `virtual`, the call is bound at compile time to the **base** version (the pointer's static type) — this contrast is the single most examined C++ topic.

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw(){ cout << "Shape\n"; }    // virtual
};
class Circle : public Shape {
public:
    void draw(){ cout << "Circle\n"; }          // override
};
int main(){
    Shape *p = new Circle();
    p->draw();          // run-time: calls Circle::draw
    return 0;
}
```

Expected output: `Circle` (with `virtual` removed, it would print `Shape`).

How it works — the vtable. For every class with virtual functions the compiler builds a **vtable** (virtual function table): an array of pointers to that class's virtual functions. Each object of such a class holds a hidden **vptr** pointing to its class's vtable. A virtual call is resolved by **following the object's vptr into the vtable** at run time — so the *actual* object's function is invoked. This is the implementation behind late binding.

18.1 Compile-time vs run-time polymorphism

	Compile-time (static)	Run-time (dynamic)
Also called	Early binding	Late binding
Achieved by	Function/operator overloading , templates	Virtual functions (via base pointer/ref)
Bound when	At compile time	At run time
Speed	Faster	Slight overhead (vtable lookup)
Flexibility	Less	More

18.2 Abstract classes and pure virtual functions

A **pure virtual function** has **no body** and is declared `= 0`:

`virtual void draw() = 0;` A class containing at least one pure virtual function is an **abstract class** — it **cannot be instantiated**; it exists only to be inherited, forcing derived classes to **provide** the implementation (an enforced interface). A derived class that fails to override all pure virtuals is itself abstract.

```
class Shape {                                // abstract
public:
    virtual double area() = 0;                // pure virtual
};
```

```
class Square : public Shape {
    double s;
public:
    Square(double x): s(x) {}
    double area(){ return s*s; }    // must override
};
```

Likely exam questions

- **[5]** What is a virtual function? Why is it needed?
- **[5]** Differentiate compile-time and run-time polymorphism.
- **[5]** What is an abstract class / pure virtual function?
- **[10]** Explain run-time polymorphism with virtual functions and a program; describe how the vtable and vptr implement dynamic binding.
- **[15]** Explain polymorphism in C++ in full: overloading, virtual functions, abstract classes, and the vtable mechanism, with programs and expected output.

MCQ traps

Trap	Correct answer
"Virtual functions enable?"	Run-time (dynamic) polymorphism.
"Late binding is implemented via?"	The vtable / vptr .
"A pure virtual function is declared with?"	= 0.
"A class with a pure virtual function is?"	Abstract — cannot be instantiated.
"Without virtual , base-pointer call binds to?"	The base version (compile time).
"Can a constructor be virtual?"	No. (A destructor should be virtual in a base class.)

19. Templates, exceptions, friends, static, **this**, inline

19.1 Templates — generic programming

A **template** lets you write a function or class **once for any data type**, the type supplied at compile time. This is **generic programming** and a form of **compile-time polymorphism**.

```
#include <iostream>
using namespace std;
template <class T>
T maxOf(T a, T b){ return (a > b) ? a : b; }
int main(){
    cout << maxOf(3, 7) << " " << maxOf(2.5, 1.5) << " " << maxOf('a','z') << endl;
    return 0;
}
```

Expected output: 7 2.5 z

Class template: `template <class T> class Stack { T data[100]; ... };`. The C++ **STL**

(vector, list, map) is built entirely on templates.

19.2 Exception handling

C++ separates **error handling** from normal logic using `try` / `throw` / `catch`: code that may fail goes in `try`; on an error it **throws** an exception object; a matching `catch` block handles it. This avoids error-code clutter and lets errors propagate up the call stack.

```
#include <iostream>
using namespace std;
int main(){
    int a = 10, b = 0;
    try {
        if(b == 0) throw "Divide by zero!";
        cout << a / b;
    } catch(const char *msg){
        cout << "Error: " << msg << endl;
    }
    return 0;
}
```

Expected output: `Error: Divide by zero!`

A `catch(...)` block catches **any** exception. Order catch blocks specific → general.

19.3 Friend functions

A **friend function** is a **non-member** function granted access to a class's **private and protected** members. Declared inside the class with the `friend` keyword but defined outside; it is **not** called on an object (no `this`). Used when a function needs the internals of **two** classes, or for operator overloading like `<<`. **Friendship is not inherited and not mutual** — examined points.

19.4 Static members

- **Static data member: one copy shared by all objects** of the class (a class-wide variable, e.g. an object counter); must be **defined outside** the class.
- **Static member function: can be called without an object** (`ClassName::func()`); can access **only static members** (it has **no `this` pointer**).

```
#include <iostream>
using namespace std;
class Counter {
    static int count;           // shared
public:
    Counter(){ count++; }
    static int get(){ return count; }
};
int Counter::count = 0;        // definition
int main(){
    Counter a, b, c;
    cout << Counter::get() << endl;    // called without an object
    return 0;
}
```

```
}
```

Expected output: 3

19.5 The `this` pointer

Every **non-static** member function receives a hidden pointer `this` holding the **address of the object** on which it was called. Used to disambiguate a member from a parameter of the same name (`this->x = x;`) and to **return the object itself** for method chaining (`return *this;`).

19.6 Inline functions

An **inline** function requests the compiler to **replace the call with the function body**, eliminating call overhead — best for **small, frequently-called** functions. `inline` is only a **request**; the compiler may ignore it (e.g. for large or recursive functions). Member functions **defined inside the class body are implicitly inline**.

Likely exam questions

- [5] What is a template? Write a template function for finding the maximum.
- [5] Explain exception handling with `try`, `throw`, `catch`.
- [5] What is a friend function? State its characteristics.
- [5] Explain static data members and static member functions.
- [5] What is the `this` pointer? Give one use.
- [10] Explain templates and exception handling in C++ with programs and expected output.

MCQ traps

Trap	Correct answer
"Templates support?"	Generic (type-independent) programming .
"Exception is raised by?"	<code>throw</code> ; handled by <code>catch</code> .
" <code>catch(...)</code> catches?"	Any exception.
"A friend function is a?"	Non-member with access to private members; no <code>this</code> .
"Static data member has how many copies?"	One , shared by all objects.
"A static member function has a <code>this</code> pointer?"	No .
" <code>this</code> pointer holds?"	The address of the current object .
" <code>inline</code> is a?"	Request to the compiler (may be ignored).

20. Last-week revision sheet

Thirty facts most likely to be one-mark MCQs:

1. **C** = Dennis Ritchie, 1972, for UNIX; middle-level, procedural. **C++** = Stroustrup, 1979.
2. Execution starts at `main()`; `#include`/`#define` handled by the **preprocessor**.

3. **32 keywords** in ANSI C; identifiers can't start with a digit; C is **case-sensitive**.
4. `int = 4 B`, `double = 8 B`, `float = 4 B`, `char = 1 B`; `sizeof` gives the exact size.
5. `5 / 2 = 2` (integer division); cast for real division.
6. Precedence: **unary** > `*` / `%` > `+` `-` > **relational** > `&&` > `||` > `?:` > `=`; unary, ternary, assignment are **right-associative**.
7. **Post-increment uses-then-increments; pre-increment increments-then-uses.**
8. `scanf` needs `&`; `getch()` = no echo, no Enter; `getchar()` echoes.
9. **do-while is the only exit-controlled loop** — runs at least once.
10. `switch` on int/char only; missing `break` = fall-through.
11. `break` exits the loop; `continue` skips to the next iteration.
12. Arrays index from 0; **no bounds checking**; a string ends in `\0`.
13. `strlen` excludes `\0`; `strcmp` returns 0 when equal.
14. **C is call-by-value**; "call by reference" = passing **pointers**.
15. Storage classes: `auto` (garbage, stack), `register` (CPU), `static` (0, persists between calls), `extern` (global).
16. `p+1` for an int pointer moves `sizeof(int)` bytes; `a[i] == *(a+i)`.
17. `malloc` = garbage, 1 arg; `calloc` = zeroed, 2 args; `realloc` resizes; `free` releases — all in `<stdlib.h>`; return **NULL** on failure.
18. **struct = sum of members; union = largest member; union shares one memory.**
19. File pointer is `FILE *`; mode `"w"` truncates; check **NULL** after `fopen`.
20. **Macro trap**: `#define SQ(x) x*x`, `SQ(2+3)=11`; parenthesise for 25.
21. **Four pillars**: encapsulation, abstraction, inheritance, polymorphism; **data hiding via private**.
22. Default access: `class` → **private**, `struct` → **public**.
23. **Constructor**: class name, no return type, auto-called; **destructor** ~, reverse order.
24. **Copy constructor takes const T&**; default copy = **shallow**.
25. **Overloading = compile-time (early) polymorphism**; can't overload by return type alone.
26. **Cannot overload**: `::?:.* sizeof`.
27. **Diamond problem** → **virtual base classes**. Inheritance = "is-a".
28. **Virtual functions** → **run-time (late) binding via the vtable/vptr**.
29. **Pure virtual = 0** → **abstract class** → **cannot be instantiated**. Destructor should be virtual; constructor cannot be.
30. **Static member = one shared copy**; static function has no `this`; `this` = address of the current object.

Descriptive-paper strategy. C/C++ reliably supplies Section A short answers and at least one Section C 15-marker in all three papers. Prepare these six as guaranteed long answers, each with a short program and its **stated expected output**:

- (a) **Pointers + dynamic memory** (`malloc/calloc/realloc/free`) with the swap example;
- (b) **Storage classes** table + the `static` counter program;
- (c) **Structures vs unions** with the `sizeof` program;
- (d) **Constructors & destructors** (all three constructors + destructor order);
- (e) **Inheritance + the diamond problem + virtual base classes**;
- (f) **Polymorphism** — overloading vs virtual functions + the vtable explanation.

Always **write the program, then predict its output line by line** — a correctly traced output scores more reliably than description, and is quick to mark.